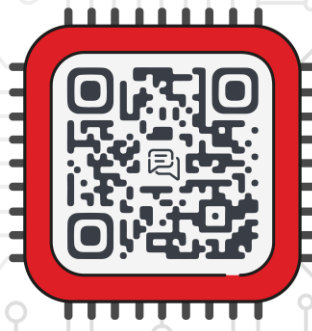




11

36

5400 sp

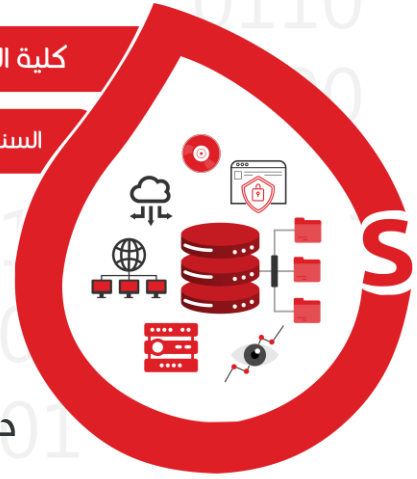
نظري

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Distributed Database

د. مادلين عبود



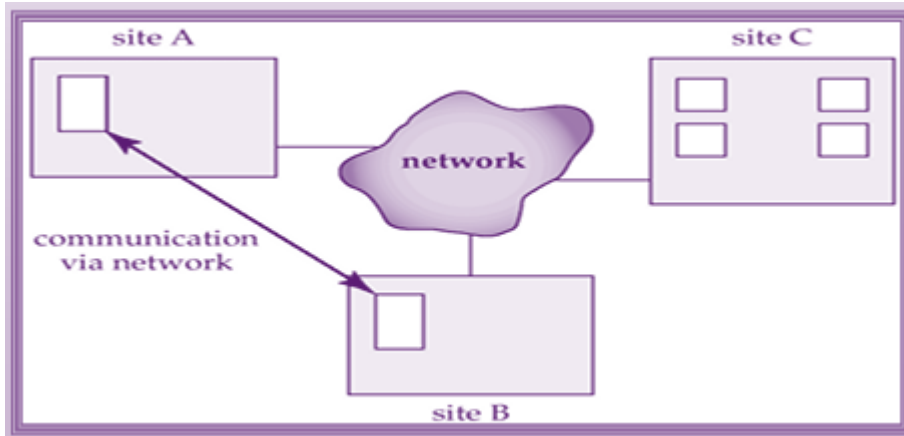
RB Informatcs; 11/12/2025

محتوى مجاني غير مخصص للبيع التجاري

قواعد معطيات ٢

في المحاضرة السابقة تحدثنا عن نظم قواعد المعطيات الموزعة وأهم النقاط فيها:

- تتعامل نظم قواعد المعطيات الموزعة مع معطيات موزعة في مواقع مستقلة فيزيائياً، وبمكونات فيزيائية متصلة بواسطة شبكة من نوع WAN.
- تكون نظم قواعد المعطيات المتواجدة في كل موقع مستقلة عن بعضها البعض.
- يمكن لمناقلة الوصول إلى معطيات متواجدة في موقع واحد أو أكثر.



قواعد المعطيات الموزعة المتجانسة

Homogeneous Distributed Databases

في هذا النوع:

- نستخدم نفس البرمجيات في جميع المواقع.
- نستخدم نفس المخططات (Schemas) لقواعد المعطيات.
- كل موقع:
 - مدرك لبقية المواقع.
 - يتعاون معها في معالجة استعلامات المستثمر والمناقلات.

أصعب معركة في حياتك لن تكون مع الآخرين، بل مع النسخة الكسولة منك

الاستقلالية:

- لكل موقع نوع من الاستقلالية الذاتية:
- يمكنه تغيير مخططه أو برمجياته ضمن حدود معينة
- رغم ذلك، النظام يبدو للمستثمر (المستخدم) كأنه:
- نظام واحد موحد وليس عدة أنظمة.

قواعد المعطيات الموزعة غير المتجانسة Heterogeneous Distributed Databases

في هذا النوع:

- المواقع المختلفة قد تستخدم:
- مخططات مختلفة
- برمجيات مختلفة

كل موقع قد يكون مصمم بطريقة مختلفة عن الآخر.

المشاكل الناتجة:

1. اختلاف المخططات:

يسبب مشكلة في معالجة الاستعلامات (كيف نفسر الاستعلام على مخططات مختلفة؟)

2. اختلاف البرمجيات:

يسبب مشكلة في معالجة المناقلات (اختلاف آليات التنفيذ والتزامن)

3. عدم إدراك المواقع لبعضها:

يقلل من التسهيلات المقدمة للتعاون في معالجة المناقلات

تخزين المعطيات الموزعة Distributed Data Storage

نفترض أن لدينا نموذج معطيات علاقتي (Relational Data Model)، أي أن المعطيات ممثلة على شكل علاقات (Relations)، ويقوم نظام قواعد المعطيات الموزعة بتخزين هذه العلاقات على عدة مواقع مستقلة فيزيائياً. يتم تخزين العلاقات الموزعة باستخدام أسلوبين رئيسيين، ويمكن الجمع بينهما:

1. تكرار المعطيات (Replication): يقوم النظام بإنشاء عدة نسخ من المعطيات المخزنة في مواقع مختلفة بهدف تسريع عملية البحث والتقليل من احتمالية الأخطاء.

2. تجزئة المعطيات (Fragmentation): العلاقة تكون مُجَرَّاة إلى عدة أجزاء مخزنة في مواقع مختلفة

- يمكن الجمع بين أسلوبَي التجزئة والتكرار في النظام نفسه.
- يقوم النظام بتجزئة العلاقة إلى عدة أجزاء، ثم يتعامل مع عدة نسخ مطابقة لكل جزء عند الحاجة

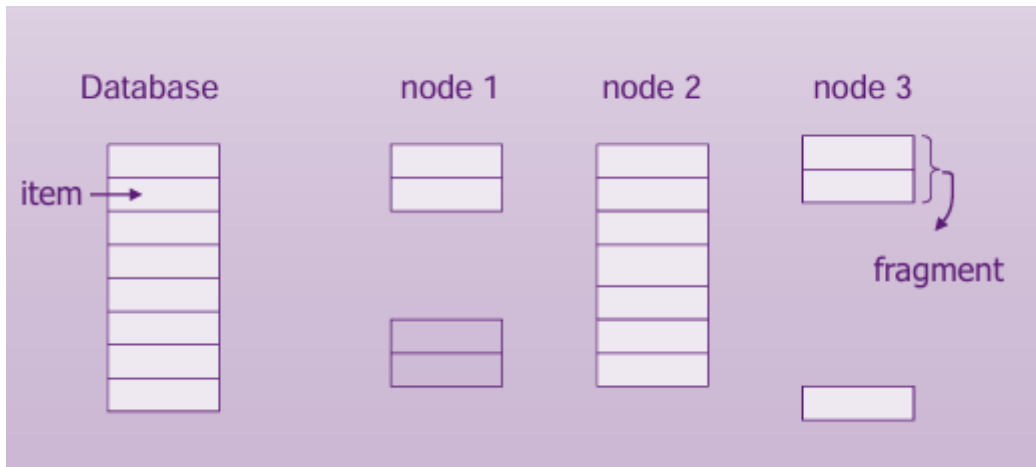
أولاً: تكرار المعطيات Data Replication

التعريف: نقول عن علاقة أو جزء من علاقة أنها مكررة إذا جرى تخزينها بشكل مكرّر في موقعين أو أكثر.

إذا جرى تخزين العلاقة كاملة ضمن عدة مواقع، فإننا نحصل على:

التكرار الكامل (Full Replication – Fully Redundant Databases)

- نقول إن لدينا تكراراً كاملاً عندما:
 - يحتوي كل موقع على نسخة كاملة من قاعدة المعطيات.
 - أي أن كل Node يمتلك نسخة مطابقة من جميع العلاقات.



الصورة تمثل قاعدة بيانات موزعة تتكوّن من عدة عقد (Nodes) متصلة معاً عبر الشبكة.

- في الجهة اليسرى نرى Database الأصلية، وهي تحتوي على مجموعة من السجلات (Items).
- على اليمين نرى node 1، node 2، node 3، وكل واحدة تمثل موقعاً فيزيائياً مختلفاً.

محاسن تكرار المعطيات:

1. الإتاحة (Availability):

- إذا وُجد تكرار للعلاقة r :
- فإن تعطل موقع يحتوي على r لا يؤدي إلى عدم توفر العلاقة.
- يمكن الوصول إليها من موقع آخر.

2. التوازي (Parallelism):

- يمكن تنفيذ استعلامات متعددة على العلاقة r:
- من قبل عدة مواقع بشكل متواز.

3. تقليل نقل المعطيات (Reduced Data Transfer):

- بما أن العلاقة متوفرة محلياً في كل موقع يحتوي على نسخة منها:
- لا حاجة لنقلها عبر الشبكة.

مسائراً تكرار المعطيات:

1. زيادة كلفة التعديل:

- أي تعديل على المعطيات يجب أن:
- يُنَفَّذَ على جميع النسخ المكررة.

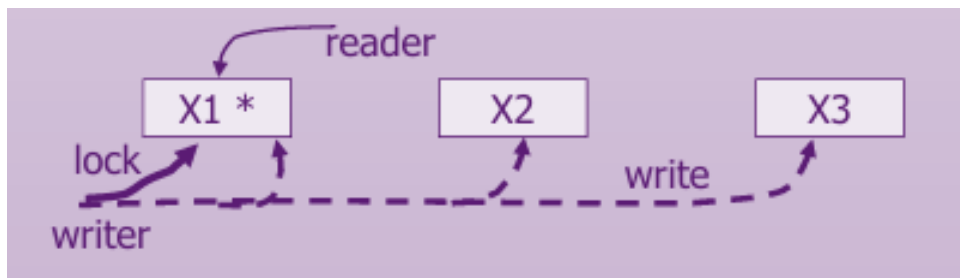
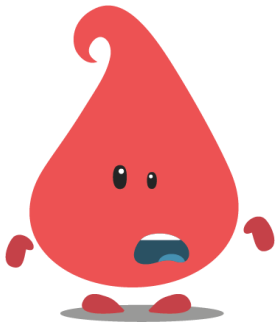
2. زيادة تعقيد التحكم بالوصول المتزامن:

- التعديل المتزامن على المعطيات المكررة قد يؤدي إلى:
- عدم اتساق قاعدة المعطيات (قاعدة معطيات غير متلائمة / inconsistent)
- وذلك في حال لم تُستخدم تقنيات خاصة للتحكم بالوصول المتزامن.

أحد الحلول المقترحة لمعالجة مشكلة التحكم بالوصول المتزامن على البيانات المكررة هو استخدام:

النسخة الأساسية Primary Copy

- يتم اختيار نسخة واحدة من المعطيات لتكون:
- نسخة أساسية
- تطبّق عليها:
- عمليات التحكم بالوصول المتزامن
- عمليات الكتابة
- ثم تُنشر التعديلات إلى بقية النسخ



ماذا يحدث في الصورة؟

1. النسخة الأساسية * X1:

- هي المكان الوحيد الذي:
 - تطبق عليه أقفال التزامن (Locks)
 - تنفذ عليه عمليات الكتابة (Write)
- لذلك نلاحظ:
 - الكاتب (writer) يطلب lock على * X1
 - هذا يضمن عدم تعارض عمليات القراءة والكتابة

2. القراء (reader):

- القراءة يمكن أن تتم:
 - من النسخة الأساسية
 - أو من النسخ الأخرى (X2, X3)
- لكن التنسيق والتزامن يبقى تحت سيطرة النسخة الأساسية

3. عملية الكتابة (Write):

- أي تعديل:
 - يتم أولاً على * X1 فقط
- بعد نجاح التعديل:
 - يتم نشر التغييرات إلى النسخ الأخرى: X2 و X3
- الخط المتقطع في الصورة يوضح انتقال التحديثات من النسخة الأساسية إلى بقية النسخ

ثانياً: تجزئة المعطيات Fragmentationالتعريف: هي عملية تقسيم العلاقة r إلى مجموعة من العلاقات الجزئية: $r_1, r_2, \dots, r_n$ بحيث:

- تحتوي هذه العلاقات الجزئية على معلومات كافية
- تسمح هذه المعلومات ب إعادة بناء العلاقة الأصلية r

من اعتاد الأعداء، سيعتاد الهزيمة دون أن يشعر

أنواع تجزئة المعطيات

1. التجزئة الأفقية Horizontal Fragmentation:

- نقول إن لدينا تجزئة أفقية عندما:
 - تقسم العلاقة إلى عدة علاقات جزئية
 - كل علاقة تحتوي على مجموعة من الصفوف
- أي كل حدودية من العلاقة r تنسب إلى علاقة جزئية أو أكثر من العلاقات الجزئية.

مثال:

علاقة الحسابات:

Account(branch-name, account-number, balance)

- الحسابات في فرع Hillside:

<i>branch_name</i>	<i>account_number</i>	<i>balance</i>
Hillside	A-305	500
Hillside	A-226	336
Hillside	A-155	62

$$account_1 = \sigma_{branch_name="Hillside"}(account)$$

- الحسابات في فرع Valleyview:

<i>branch_name</i>	<i>account_number</i>	<i>balance</i>
Valleyview	A-177	205
Valleyview	A-402	10000
Valleyview	A-408	1123
Valleyview	A-639	750

$$account_2 = \sigma_{branch_name="Valleyview"}(account)$$

2. التجزئة العمودية Vertical Fragmentation:

- نقول إن لدينا تجزئة عمودية عندما:
 - تقسم العلاقة إلى عدة علاقات ذات مخططات أصغر
 - كل علاقة تحتوي على جزء من الصفات (جزء من الأعمدة)

شروط هامة في التجزئة العمودية:

1. يجب أن تحتوي جميع المخططات الجزئية على:
 - مفتاح مشترك (مفتاح أساسي أو مفتاح مرشح) == (ليمكننا من الربط بين الأجزاء)
2. يجب تحقيق خاصية:
 - Lossless Join Property خاصة الدمج المحافظ على المعطيات
3. يمكن استخدام:
 - واصفة خاصة tuple-id (رقم الحدودية) لكل مخطط جزئي
 - وذلك لتسهيل الدمج لاحقاً

مثال:

علاقة employee_info

employee_info _schema = (branch-name, customer_name, account-number, balance)

يتم تجزئتها إلى:

deposit1

يحتوي على الأعمدة:

branch_name

customer_name

tuple_id

هذا الجزء يمثل معلومات الفرع والعميل فقط.

deposit2

يحتوي على الأعمدة:

account_number

balance

tuple_id

هذا الجزء يمثل معلومات الحساب والرصيد فقط.

branch_name	customer_name	tuple_id
Hillside	Lowman	1
Hillside	Camp	2
Valleyview	Camp	3
Valleyview	Kahn	4
Hillside	Kahn	5
Valleyview	Kahn	6
Valleyview	Green	7

$$deposit_1 = \Pi_{branch_name, customer_name, tuple_id}(employee_info)$$

account_number	balance	tuple_id
A-305	500	1
A-226	336	2
A-177	205	3
A-402	10000	4
A-155	62	5
A-408	1123	6
A-639	750	7

$$deposit_2 = \Pi_{account_number, balance, tuple_id}(employee_info)$$

ويتم إعادة بناء العلاقة باستخدام join على tuple-id.

فائدة أربيسيزية:

خاصية إعادة التركيب بدون فقدان/الدمج المحافظ على المعطيات (Lossless Join Property) تعني أن العلاقة الأصلية يمكن استعادتها بالكامل عند إعادة دمج الأجزاء الممزقة، من دون فقدان أي معلومة أو ظهور بيانات خاطئة.

بمعنى آخر:

إذا قمنا بتجزئة جدول إلى أجزاء، ثم أعدنا جمع هذه الأجزاء، يجب أن نحصل على نفس الجدول الأصلي تماماً.

محاسن التجزئة:

التجزئة الأفقية:

- تسمح بالمعالجة المتوازية على أجزاء العلاقة (أي أن كل جزء من العلاقة يمكن معالجته في نفس الوقت على موقع مختلف أو خادم مختلف، بدلاً من معالجة العلاقة/الجدول كاملاً في مكان واحد).
- تسمح بتخزين الصفوف الأكثر استخداماً في المواقع المناسبة لها.

التجزئة العمودية:

- تسمح بتخزين أجزاء من الحدوديات الأكثر استخداماً في المواقع المناسبة.
- تسمح بدمج الأجزاء العمودية بكفاءة باستخدام tuple-id.
- تدعم المعالجة المتوازية.

الدمج بين التجزئة الأفقية والعمودية:

يمكن دمج:

- التجزئة الأفقية
- والتجزئة العمودية

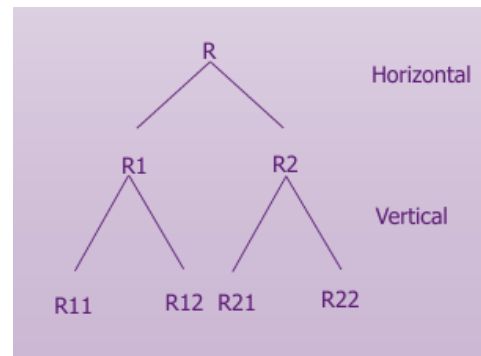
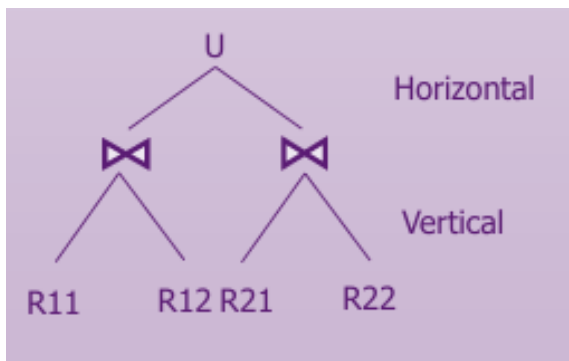
ويمكن تنفيذ التجزئة:

- بشكل هرمي (شجري) (تسمى تجزئة شجرية)

مثال:

- تجزئة أفقية أولاً

- ثم تجزئة عمودية لكل جزء (أو العكس)



شفافية المعطيات Data Transparency

التعريف: تعكس شفافية المعطيات الدرجة التي يمكن عندها أن يبقى مستثمر النظام غير معنيّ بتفاصيل:

- كيفية تخزين المعطيات
- مكان تخزين المعطيات
- كون المعطيات موزعة أو لا

في نظام قواعد المعطيات الموزعة، يجب أن يبدو النظام للمستثمر كنظام واحد مركزي، رغم أن المعطيات موزعة فعلياً على عدة مواقع.

أنواع شفافية المعطيات:

يمكن إرجاع شفافية المعطيات إلى الأنواع التالية:

1. شفافية التجزئة Fragmentation Transparency:

- لا يشعر المستثمر بأن العلاقة:
 - مجزأة أفقياً
 - أو مجزأة عمودياً
- يستطيع التعامل مع العلاقة وكأنها:
 - علاقة واحدة غير مجزأة
- يقوم النظام تلقائياً ب:
 - الوصول إلى الأجزاء المناسبة
 - إعادة دمج النتائج

2. شفافية التكرار Replication Transparency:

- لا يشعر المستثمر بوجود:
 - نسخ مكررة من المعطيات
- لا يحتاج لمعرفة:
 - أي نسخة يتم الوصول إليها
- النظام يختار:
 - النسخة الأنسب من حيث الأداء والتوفر

3. شفافية مكان التخزين Location Transparency:

- لا يحتاج المستثمر إلى معرفة:
 - الموقع الفيزيائي الذي تُخزن فيه المعطيات
- يمكن نقل المعطيات من موقع إلى آخر:
 - دون التأثير على الاستعلامات

تسمية عناصر المعطيات Naming of Data Items

لكي نتحقق شفافية المعطيات، لا بد من وجود آلية تسمية فعالة لعناصر المعطيات.

⚡ شروط تسمية عناصر المعطيات Naming Criteria:

1. يجب إعطاء اسم وحيد لكل عنصر من المعطيات على مستوى النظام الموزع.
2. إمكانية إيجاد مكان العنصر بفعالية.
3. إمكانية تغيير مكان تخزين العنصر بشفافية.
4. يستطيع كل موقع:
 - إضافة عناصر معطيات جديدة خاصة به (إعطاء تسمية).

استخدام مخدم تسمية مركزي Central Name Server

الفكرة:

- يوجد مخدم تسمية مركزي:
 - يقدم أسماء لجميع عناصر المعطيات.
- كل موقع:
 - يتعامل مباشرة مع عناصر المعطيات المحلية.
 - يطلب من مخدم التسمية الوصول إلى العناصر غير المحلية.

البنية:

- جميع العمليات:
 - تسمية العناصر
 - والوصول إليها
- ← تمر عبر مخدم التسمية.

الحسنات:

- تحقيق خصائص التسمية بشكل واضح (شروط تسمية عناصر المعطيات من 1 إلى 3).

المساوئ:

- عنق الزجاجة (Bottleneck):
 - يؤثر سلباً على الأداء (لأن جميع عمليات تسمية العناصر والوصول إليها يمر من خلاله).
- تعطل مخدم التسمية يؤدي إلى:
 - تعطل النظام بأكمله
- لا تحقق الخاصية:
 - التسمية اللامركزية (شروط التسمية رقم 4 من شروط تسمية عناصر المعطيات التي ذكرناها سابقاً).

استخدام المترادفات Use of Aliases

الفكرة:

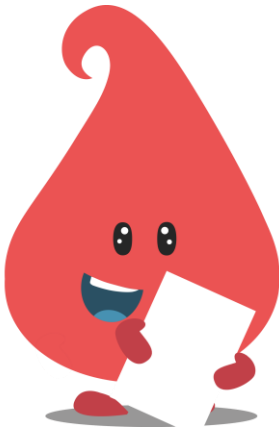
- كبدل عن المخدم المركزي:
- يقوم كل موقع بإضافة **مميز خاص به** في بداية أسماء العناصر التي يولدها.
- مثال: site17.account

الهدف:

- تجنب مشاكل التسمية المرتبطة بالمخطط المركزي.
- استخدام **مميز وحيد** لكل موقع:
- لجميع العناصر المولدة منه.

المساوئ:

- فشل في تحقيق شفافية الشبكة:
- لأن مميز الموقع يصبح جزءاً من اسم العنصر.
- هذا يعني التعامل المباشر مع:
- المكان الفيزيائي للعنصر



استخدام المترادفات (use of aliases) كحل لمشكلة ارتباط أسماء العناصر

بمواقعها الفيزيائية:

- يتم بناء مجموعة مترادفات (Aliases):
- تربط بين:
 - الأسماء الحقيقية
 - والأسماء البديلة
- يتم تخزين:
 - طريقة التحويل بين المرادفات والأسماء الحقيقية في كل موقع.
- باستخدام المترادفات:
 - يتجنب المستثمر التعامل مع: أسماء المواقع (التعامل مع المكان الفيزيائي للعنصر)
 - وبذلك: لا يتأثر نقل المعطيات من موقع إلى آخر.

لا تنتظر الإذن لتكون عظيماً، فالوقت يمضي، والفرص تفلت؛ إما أن تصنع

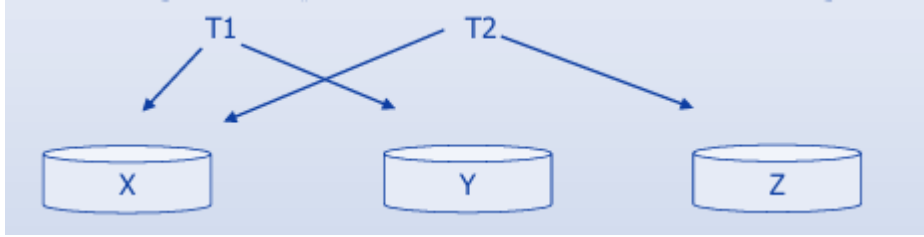
مستقبلك اليوم، أو تشاهد الآخرين يصنعونه بدلاً عنك

المناقلات الموزعة

Distributed Transactions

التعريف: المناقلة الموزعة هي مناقلة واحدة قد تحتاج إلى الوصول إلى معطيات موجودة في عدة مواقع ضمن نظام قواعد معطيات موزعة.

أي أن تنفيذ المناقلة لا يتم في موقع واحد فقط، بل يشمل عدة مواقع تشترك جميعها في إنجازها.



مكونات إدارة المناقلات الموزعة:

1. مدير المناقلات Transaction Manager:

يوجد في كل موقع، وهو مسؤول عن:

- إدارة المناقلات المحلية
- التعامل مع ملف التسجيل (Log File) لأغراض: الاستعادة (Recovery)
- (Coordinating) المشاركة في تنسيق الوصول المتزامن للمناقلات التي تعمل في الموقع.
- تنفيذ القرارات:
 - التثبيت (commit)
 - أو الاستبعاد (abort)

2. منسق المناقلات Transaction Coordinator:

يوجد أيضاً في كل موقع، وهو مسؤول عن المناقلات التي تبدأ من هذا الموقع.

مهامه:

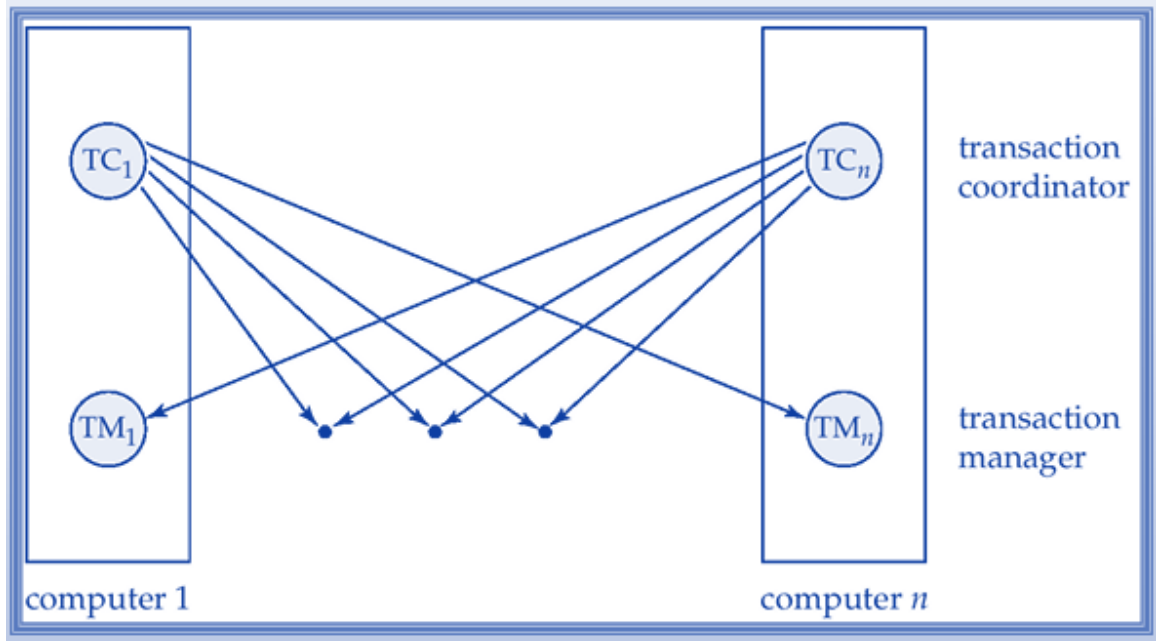
- إقلاع تنفيذ المناقلة.
- توزيع المناقلات الجزئية على المواقع المعنية للتنفيذ (Appropriate Sites).
- تنسيق نهاية كل مناقلة بدأت في الموقع:
 - إما تثبيت المناقلة في جميع المواقع
 - أو استبعادها في جميع المواقع

أي أنه غير مقبول أن:

- تثبت المناقلة في موقع
- وتستبعد في موقع آخر

معمارية نظام المناقلات

Transaction System Architecture



بالإضافة إلى الأعطال التقليدية في الأنظمة المركزية (أعطال برمجية، أعطال تجهيزات)، تظهر في النظام الموزع أعطال خاصة، منها:

أنواع الأعطال في النظام الموزع:

1. عطل موقع

- توقف موقع كامل عن العمل.

2. ضياع الرسائل

- نتيجة مشاكل في بروتوكولات النقل الشبكي مثل TCP/IP.

3. تقسيم الشبكة (Network Partition):

نقول عن شبكة أنها مقسمة عندما:

- تنقسم الشبكة إلى قسمين أو أكثر دون وجود اتصال بينها.
- أعطال تقسيم الشبكة وأعطال المواقع غالباً غير متميزة (indistinguishable).
- معنى غير متميزة:** يعني أن النظام لا يستطيع دائماً التمييز بين الحالتين:
 - هل الموقع فعلاً تعطل؟
 - أم أن الموقع يعمل لكن الشبكة بينه وبين بقية النظام متقسمة؟

من منظور النظام، النتيجة واحدة: وهي أن "هذا الموقع لا يستجيب"

بروتوكولات التثبيت

Commit Protocols

تستخدم بروتوكولات التثبيت للتأكد من الكتلية (Atomicity) عند تنفيذ المناقلات عبر عدة مواقع.

الهدف:

- المناقلة يجب أن:
- تثبت في جميع المواقع
- أو تستبعد في جميع المواقع

↪ بروتوكول التثبيت على مرحلتين 2PC – Two Phase Commit Protocol

هو البروتوكول الأكثر استخداماً (مستخدم بشكل واسع).

النموذج المفترض للأعطال:

- Fail-stop model
- الموقع الذي يفشل يتوقف عن العمل كلياً
- ولا يرسل رسائل خاطئة الى المواقع الأخرى.

■ بروتوكول التثبيت لا يبدأ إلا بعد انتهاء تنفيذ جميع خطوات المناقلة نفسها

أي: يقلع البروتوكول بالتنفيذ بطلب من المنسق (coordinator) بعد الوصول الى الخطوة الأخيرة من المناقلة (هدف البروتوكول هو تثبيت الخطوات أو التراجع)

- كل موقع شارك في تنفيذ جزء من المناقلة يجب أن يشارك أيضاً في بروتوكول التثبيت.

ليكن:

- المناقلة T بدأت في الموقع Si
- المنسق المسؤول عن هذه المناقلة هو Ci الموجود في نفس الموقع Si

الطور الأول: الحصول على قرار

Phase 1: Obtaining a Decision

1. Coordinator asks all participants to prepare to commit transaction T

يطلب المنسق Ci من جميع المواقع المشاركة الاستعداد لتثبيت المناقلة T.

2. Ci adds the records <prepare T> to the log and forces log to stable storage

يسجل المنسق في ملف التسجيل log file : <prepare T>

ويجبر LOG FILE على التخزين المستقر (إجبار السجل على الحفظ في الذاكرة الدائمة (stable storage) لضمان عدم فقدان القرار في حال حدوث عطل)

3. Sends < prepare T > messages to all sites at which T executed

يرسل رسالة < prepare T > إلى جميع المواقع التي تنفذ فيها المناقشة T.

عند وصول الرسالة إلى كل موقع مشارك:

كل موقع يحتوي على Transaction Manager يقوم بتحديد ما إذا كان قادراً على تثبيت المناقشة.

■ إذا كان التثبيت غير ممكن:

- يسجل < no T > الى log file
- ويرسل رسالة abort T إلى المنسق ci

■ إذا كان التثبيت ممكن:

- يسجل < ready T > في السجل (log file)
- يجبر كل سجلات المناقشة على الحفظ في الذاكرة الدائمة (في تخزين مستقر)
- يرسل رسالة ready T إلى المنسق Ci

الطور الثاني: تسجيل القرار

Phase 2: Recording the Decision

■ إذا تلقى المنسق ready T من جميع المواقع المشاركة في تنفيذ المناقشة:

- يقرر تثبيت المناقشة.

■ في غير ذلك:

- يقرر استبعاد المناقشة.

يسجل المنسق القرار النهائي: < commit > أو < abort T > إلى log file
ويجبر على وضع التسجيلات في حالة تخزين مستقر (عندها يكون التسجيل نهائي)

ثم:

■ يرسل القرار المتخذ إلى جميع المواقع.

■ تقوم المواقع بتنفيذ العمل محلياً:

- redo(T) عند التثبيت
- undo(T) عند الاستبعاد

القرار النهائي لا يمكن إلغاؤه حتى في حال حدوث عطل.

التعامل مع الأعطال Handling of Failures

أولاً: عطل موقع Site Failure

1. إذا اكتشف المنسق Ci حدوث عطل في موقع مشارك ينظر إلى وقت حدوث العطل وهنا تظهر حالتان:

الحالة الأولى: العطل حدث قبل أن يرسل الموقع ردّه

إذا تعطل الموقع قبل إرسال أي رد على رسالة prepare:

- يعتبر المنسق أن رد الموقع سلبي
- وبالتالي يتخذ قرار إلغاء المناقلة (abort T) لأن الموقع لم يؤكد استعداده للتثبيت.

الحالة الثانية: العطل حدث بعد أن أرسل الموقع رسالة ready

إذا كان الموقع قد أرسل ready T ثم تعطل:

- يتجاهل المنسق العطل
- ويستمر في تنفيذ البروتوكول لأن الموقع سبق أن أعلن أنه جاهز للتثبيت.

2. عند استعادة الموقع Si بعد العطل (When site Si recovers):

عندما يعود الموقع للعمل، يقوم بفحص ملف السجل (log file) لمعرفة مصير المناقلات الفعالة وقت العطل.

- إذا وجد في السجل <commit T> ← يجب تنفيذ redo(T) أي إعادة تنفيذ التثبيت،
- إذا وجد <abort T> ← يجب تنفيذ undo(T) أي التراجع عن المناقلة.
- إذا وجد <ready T> ← هذا يعني أن الموقع كان جاهزاً للتثبيت لكنه لم يعرف القرار النهائي. لذلك يجب عليه:

- الاتصال بالمنسق Ci
- وسؤاله عن القرار النهائي للمناقلة T

ثم:

- إذا كانت المناقلة قد نُبِتت = ينفذ redo(T)
 - إذا كانت قد أُلغيت = ينفذ undo(T)
 - إذا لم يجد الموقع أي سجل متعلق بالمناقلة T ، هذا يعني:
 - أن الموقع Si تعطل قبل أن يرسل رده على رسالة prepare المرسله من قبل المنسق Ci
 - وبالتالي لم يكن مشاركاً فعلياً في اتخاذ القرار
- في هذه الحالة:

- لا يمكن للموقع إرسال رد للمنسق
- والمنسق سيقوم بإلغاء المناقلة undo(T) لأنه لم يحصل على موافقة هذا الموقع

ملخص للحالات:

الإجراء	محتوى Log
تنفيذ redo(T)	<commit T>
تنفيذ undo(T)	<abort T>
سؤال المنسق عن القرار	<ready T>
تنفيذ undo(T)	لا يوجد سجل

ثانياً: عطل المنسق Coordinator Failure

نفترض أن المنسق Ci تعطل أثناء تنفيذ بروتوكول التثبيت (2PC) للمناقلة T. في هذه الحالة، المواقع المشاركة (participants) يجب أن تحدد مصير المناقلة T بنفسها، اعتماداً على ما هو موجود في ملفات السجل (log files).

الحالات الممكنة:

الحالة (1):

إذا كان أحد المواقع الفعالة يحتوي في الـ log على: $\langle \text{commit } T \rangle$ هذا يعني أن قرار التثبيت قد اتخذ مسبقاً
 يجب تثبيت المناقلة (commit) T في جميع المواقع

الحالة (2):

إذا كان أحد المواقع الفعالة يحتوي في الـ log على: $\langle \text{abort } T \rangle$ هذا يعني أن قرار الإلغاء قد اتخذ
 يجب إلغاء المناقلة (abort) T في جميع المواقع

الحالة (3):

إذا وُجد موقع مشارك فعال لا يحتوي في الـ log على: $\langle \text{ready } T \rangle$ هذا يعني أن:
 ■ هذا الموقع لم يصل إلى مرحلة الاستعداد
 ■ بالتالي المنسق لم يكن قد قرر التثبيت
 يمكن إلغاء المناقلة T بأمان

حالة مهمة:

عدم وجود $\langle \text{ready } T \rangle$ في بعض موقع = المنسق لم يقرر تثبيت المناقلة قبل عطله وبالتالي يمكن استبعاد المناقلة

الحالة (4):

إذا تحققت الشروط التالية معاً:

- جميع المواقع الفعالة تحتوي على: $\langle \text{ready } T \rangle$
- ولا يوجد أي سجل: $\langle \text{commit } T \rangle$ أو $\langle \text{abort } T \rangle$

هنا:

- جميع المواقع وافقت على التنفيذ
- لكن قرار المنسق غير معروف
- والمنسق متعطل

يجب على المواقع الانتظار حتى يتم تغطية أو تصحيح المنسق لاتخاذ القرار

هذه الحالة تؤدي إلى:

مشكلة العرقلة (Blocking Problem): ناتجة عن أن المواقع الفعالة مضطرة لانتظار تصحيح أو تغطية المنسق المعطل

ثالثاً: عطل الشبكة Network Partition

Network Partition

- إذا كان المنسق والمواقع المشاركة في نفس القسم:
- لا يتأثر بروتوكول التثبيت بعطل الشبكة الذي أدى إلى تقسيمها.
- إذا كان المنسق في قسم والمواقع في أقسام أخرى:
- تعتبر المواقع أن المنسق معطل
- وتتعامل مع الحالة كعطل منسق (تقلع بروتوكول التعامل مع عطل المنسق)

الاستعادة والتحكم بالوصول المتزامن

Recovery and Concurrency Control

- لاحظنا أنه قد توجد مناقلات:
- تحتوي على $\langle ready T \rangle$
- ولا تحتوي على $\langle commit T \rangle$ أو $\langle abort T \rangle$
- تسمى هذه:
- مناقلات مشكوك بها In-doubt transactions
- وتحدثنا في الفقرة السابقة أنه في هذه الحالة:
- لا يستطيع الموقع أن يقرر محلياً:
- هل يقوم بتثبيت المناقلة؟
- أم ب إلغاؤها؟
- لذلك يجب عليه الاتصال بالمواقع الأخرى أو بالمنسق لمعرفة مصير المناقلة

لكن هذا يؤدي إلى:

- إبطاء عملية الاستعادة

الحل المقترح (لتحسين آلية الاستعادة):

- بدلاً من تسجيل: $\langle ready T \rangle$
- نقوم بتسجيل: $\langle ready T, L \rangle$
- حيث:
- L = قائمة الأقفال التي أخذتها المناقلة T
- ويتم تسجيل ذلك في log file قبل تثبيت المناقلة

↪ العلاقة بين الاستعادة والأقفال:

أثناء تنفيذ المناقلة:

- يتم أخذ أقفال (Locks) على عناصر المعطيات
- هذه الأقفال تبقى فعّالة إذا كانت المناقلة في حالة ready

المشكلة:

- لا يمكن تحرير الأقفال
 - ولا يمكن السماح لمناقلات جديدة باستخدام هذه العناصر
- ← هذا يؤثر سلباً على التحكم بالوصول المتزامن

↪ ماذا نستفيد عند تطبيق الحل المقترح؟

أثناء الاستعادة:

1. نعيد فحص الـ log
 2. لكل مناقلة مشكوك بأمورها:
- نعرف ما هي الأقفال التي كانت محبوزة
 - 3. بعد تحديد مصير المناقلة (تثبيت أو عودة):
 - يمكن تحرير الأقفال مباشرة
 - دون الحاجة للانتظار الطويل

توضيح: ماذا كان يحدث قبل تطبيق الحل المقترح؟

لدينا مناقلة T وصلت إلى المرحلة: <ready T>
لكن لم يُسجَل: <commit T> ولا <abort T>

يعني:

- المناقلة شاركت في بروتوكول التثبيت
- وأخذت أقفال (Locks)
- لكن القرار النهائي غير معروف

ماذا يفعل الموقع عند الاستعادة؟

1. الموقع يقرأ الـ log file
 2. يجد: <ready T>
 3. لا يجد أي قرار نهائي
- ← إذن هذه مناقلة مشكوك بأمورها

أين المشكلة هنا؟

المشكلة هي ان الموقع لا يعرف:

- هل يجب تنفيذ:
- تثبيت أم عودة



■ متى يمكنه تحرير الأقفال

■ ولا حتى ما هي الأقفال التي أخذتها المناقلة

لأن: ال log يحتوي فقط على $\langle ready T \rangle$ ولا يحتوي أي معلومات عن الأقفال

ماذا يحصل للأقفال؟

الأقفال التي أخذتها المناقلة تبقى محجوزة

■ لا يمكن تحريرها

■ لا يمكن إعطاء هذه الأقفال لمناقلات أخرى

■ المناقلات الجديدة التي تحتاج هذه العناصر:

• تنتظر أو تعلق

هذا يؤدي إلى تجميد جزئي للنظام

ماذا يفعل الموقع إذا؟

ليس أمامه إلا حل واحد وهو انتظار:

• عودة المنسق

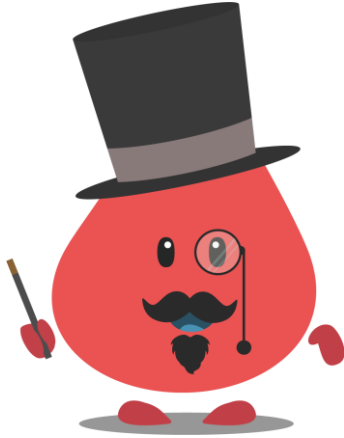
• أو تواصل مع مواقع أخرى لمعرفة القرار

■ خلال هذا الانتظار:

• الأقفال تبقى محجوزة

• التحكم بالوصول المتزامن يتدهور

وهذا ما نسميه عملياً: Blocking problem



بروتوكول التثبيت على ثلاث مراحل

Three Phase Commit – 3PC

■ بروتوكول Three-Phase Commit (3PC) هو نسخة مطوّرة من بروتوكول Two-Phase Commit (2PC)،

صُمم لتقليل مشكلة العرقلة (Blocking) التي تحدث عندما يتعطل المنسق في 2PC، مما يجعل المواقع الأخرى تنتظر عودته.

■ 3PC يضيف مرحلة إضافية بين مرحلتين 2PC لجعل النظام أكثر قدرة على اتخاذ قرار دون انتظار المنسق.

الفرضيات:

■ لا يوجد تقسيم في الشبكة.

■ يوجد دائماً على الأقل موقع فعال.

■ يمكن أن يتعطل على الأكثر K موقعاً.

الراحة التي تأتي قبل التعب، غالباً تأتي معها الندامة

➤ مراحل PC3:

المرحلة الأولى: الحصول على قرار مبدئي

- مطابقة تماماً للطور الأول في 2PC

المرحلة الثانية: (Pre-Commit)

وهذه هي المرحلة الإضافية التي تميّز 3PC.

- المنسق يصدر قرار تثبيت مبدئي/قرار التثبيت المسبق Pre-commit decision. و هذا القرار يصنع بنفس الطريقة في 2PC (إذا وافق جميع المشاركين، يقوم المنسق بإصدار قرار مبدئي بالتثبيت)
- يُسجّل القرار في عدة مواقع (على الأقل K مواقع).

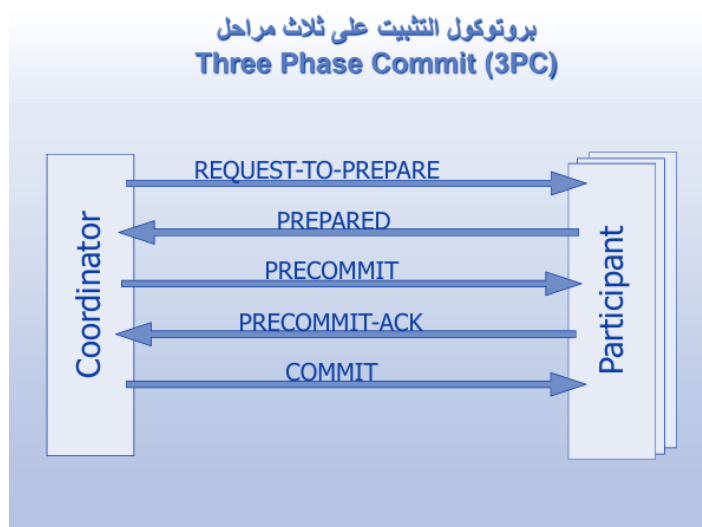
المرحلة الثالثة:

- إرسال قرار:
 - التثبيت
 - أو الاستبعاد
- حتى في حال تعطل المنسق:
 - يمكن للمواقع إكمال القرار
 - طالما عدد الأعطال أقل من K

بذلك:

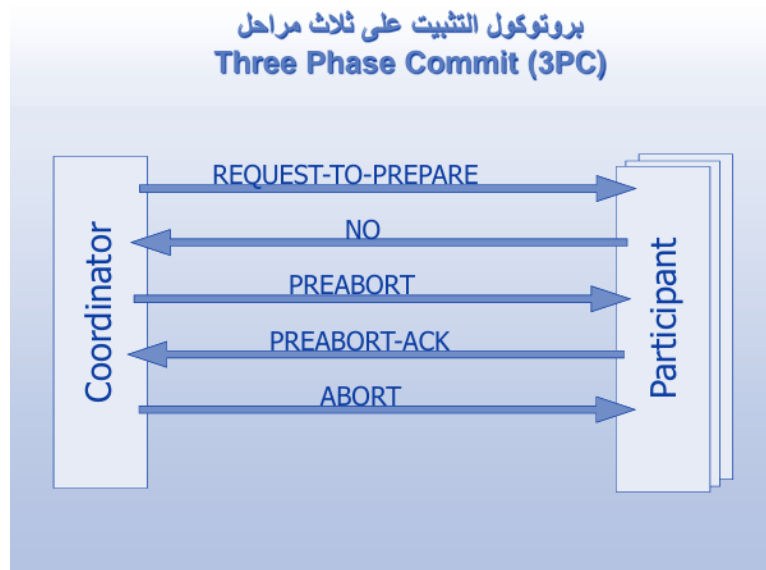
- يتم تجنب Blocking Problem

تسلسل الرسائل في حالة التثبيت (COMMIT Path):



النجاح يبدأ حين تتوقف عن انتظار التحفيز وتبدأ بالانضباط

تسلسل الرسائل في حالة الإلغاء (Abort Path):



التحكم بالوصول المتزامن في قواعد المعطيات الموزعة

Concurrency Control in Distributed DB

التحكم بالوصول المتزامن Concurrency Control :

في الأنظمة الموزعة لا يمكن استخدام مخططات التحكم بالوصول المتزامن التقليدية كما هي، بل يجب تعديلها لتناسب المحيط الموزع.

الافتراضات الأساسية:

- كل موقع مشارك في تنفيذ بروتوكول التثبيت يؤمن:
- الكتلية العامة للمناقلة Global Transaction Atomicity
- نفترض أن جميع النسخ المتوفرة للعنصر المعدل يجب أن تكون متوافقة.

اتجاه مدير أقفال واحد

Single Lock Manager Approach

الفكرة:

يتعامل النظام مع مدير أقفال واحد فقط، موجود في موقع مختار.

آلية العمل:

- عندما تحتاج مناقلة إلى قفل عنصر معطيات:
- ترسل رسالة طلب القفل إلى مدير الأقفال.

يقوم مدير الأقفال بتحديد:

• هل يمكن منح القفل؟

• نعم ← يرسل موافقة إلى موقع المناقشة

• لا ← يبقى الطلب في حالة الانتظار

• عند توفر القفل:

• يرسل المدير رسالة إلى الموقع الطالب لإعلامه بإمكانية تلبية الطلب.

التعامل مع المعطيات المكررة

• القراءة: يمكن لأي موقع قراءة المعطيات من أي نسخة مكررة.

• الكتابة: يجب تنفيذها على جميع النسخ المكررة.

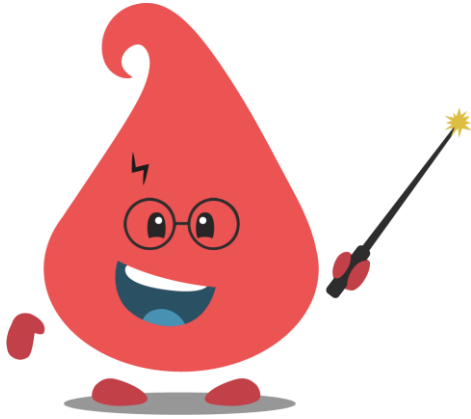
المحاسن:

• سهولة التحقيق والتنفيذ.

المساوئ:

• يشكّل موقع مدير الأقفال عنق زجاجة.

• النظام غير مقاوم لأعطال موقع مدير الأقفال.



مدير أقفال موزع

Distributed Lock Manager

الفكرة:

بدلاً من مدير أقفال واحد:

• يوجد مدير أقفال في كل موقع.

• كل مدير يتحكم بالوصول إلى المعطيات المحلية.

الخصائص:

• استخدام بروتوكول خاص للتعامل مع المعطيات المكررة

• النظام:

• أكثر مقاومة للأعطال

• العمل موزع

التحديات:

• اكتشاف العرقلة المتبادلة يصبح أصعب.

• يحتاج مدراء الأقفال إلى التعاون لاكتشاف العرقلة.

من لم يتعلّم الوقوف وحده، سيسقط كلما غاب عنه الدعم

بروتوكولات مستخدمة للتحكم في إدارة الأقفال:

- النسخة الرئيسية (Primary Copy)
- البروتوكول المعتمد على الأغلبية (Majority Protocol)
- البروتوكول المنحاز (Biased Protocol)

النسخة الرئيسية

Primary Copy Protocol

الفكرة:

- يتم اختيار نسخة واحدة من كل عنصر معطيات مكرر لتكون: النسخة الرئيسية.
- الموقع الذي يحتوي النسخة الرئيسية يسمى: الموقع الرئيسي للعنصر.

آلية العمل:

- عند طلب قفل عنصر Q: يُرسل الطلب إلى الموقع الرئيسي لـ Q.
- الحصول على القفل على النسخة الرئيسية يعني: الحصول الضمني على القفل على جميع النسخ.

في حال عطل الموقع الرئيسي:

- يصبح العنصر Q غير قابل للوصول.
- رغم وجود نسخ مكررة في مواقع أخرى.

الفائدة:

- طريقة بسيطة وسهلة التنفيذ.

السلبيّة (Drawback):

- الاعتماد الكامل على الموقع الرئيسي.
- ضعف التوافر عند الأعطال.

البروتوكول المعتمد على الأغلبية

Majority Protocol

الفكرة:

- لكل موقع مدير أقفال محلي.
- يتم منح القفل فقط إذا حصلت المناقشة على موافقة: أغلبية النسخ المكررة للعنصر.

آلية العمل:

- إذا كان العنصر Q مكرراً في n مواقع: يجب إرسال طلب القفل إلى أكثر من $n/2$ مواقع
- لا تُنفذ المناقشة إلا بعد: الحصول على أقفال الأغلبية
- عند الكتابة: تُنفذ على جميع النسخ المكررة.

الفائدة:

- يمكن استخدام البروتوكول حتى لو كانت بعض المواقع غير فعالة.

البروتوكول المنحاز

Biased Protocol

الفكرة:

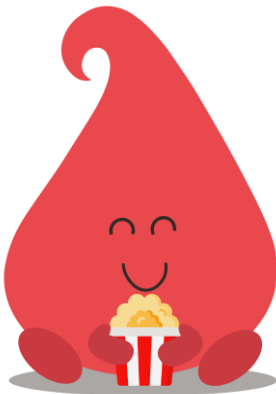
مشابه لبروتوكول الأغلبية، لكنه يميز بين نوع القفل:

الأقفال المشتركة (Shared Locks)

- يكفي طلب القفل من موقع واحد يحوي نسخة مكررة.

الأقفال الكلية (Exclusive Locks)

- يجب طلب القفل من جميع مدراء الأقفال
- على جميع المواقع التي تحوي نسخاً مكررة.

الفائدة:

- كلفة أقل في عمليات القراءة.

المساوئ:

- كلفة أعلى في عمليات الكتابة.

العرقلة المتبادلة

Deadlock Handling

مثال:

لدينا مناقلتان:

write(Y) ثم write(X) :T1

write(X) ثم write(Y) :T2

ما يحدث:

T1 تحصل على X-lock على X

T2 تحصل على X-lock على Y

T1 تنتظر Y

T2 تنتظر X

عرقلة متبادلة

المشكلة في النظام الموزع:

- لا يمكن اكتشاف العرقلة محلياً في كل موقع.

الاتجاه المركزي لاكتشاف العرقلة

Centralized Deadlock Detection

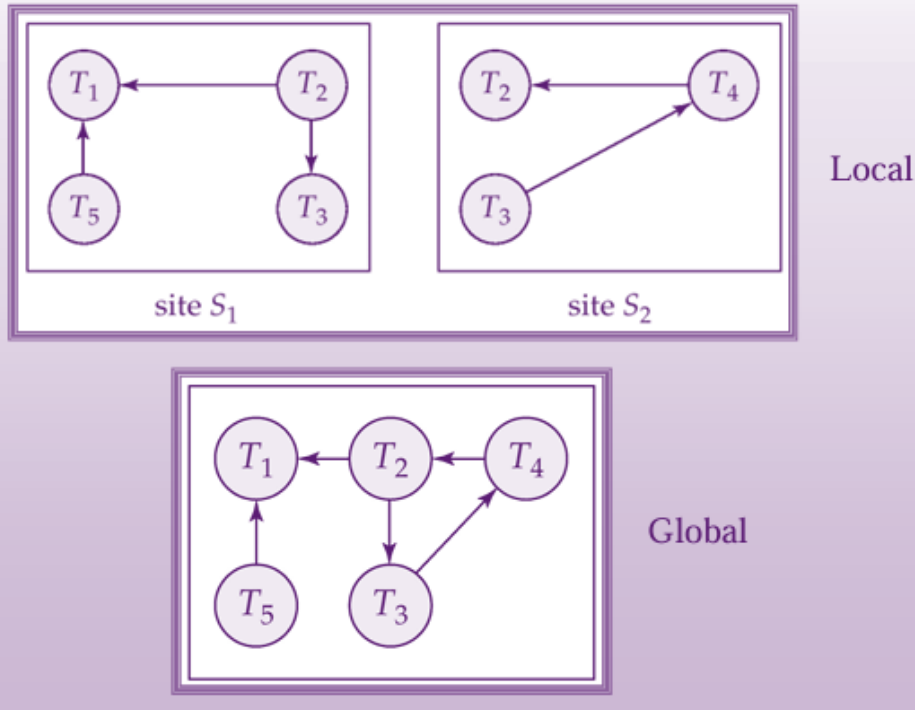
الفكرة:

- يوجد منسق واحد مسؤول عن اكتشاف العرقلة.
- يقوم ببناء:
- بيان الانتظار العام (Global Wait-For Graph)

آلية العمل:

- تستقبل جميع المواقع معلومات الانتظار.
- إذا وجد المنسق حلقة في البيان:
- يختار مناقلة كضحية
- يرسل أمر: (rollback) عملية العودة إلى جميع المواقع.

Local and Global Wait-For Graphs



الإنسان لا يتغير حين يفهم، بل حين يعترف

بيانات الانتظار المحلية والعامة

Local and Global Wait-For Graphs

:Local WFG

- يعبّر عن الانتظار داخل موقع واحد.

:Global WFG

- دمج جميع البيانات المحلية.
- يُستخدم لاكتشاف العرقلة المتبادلة الموزعة.

التحكم بالوصول المتزامن المعتمد على الأختام Timestamping

الفكرة:

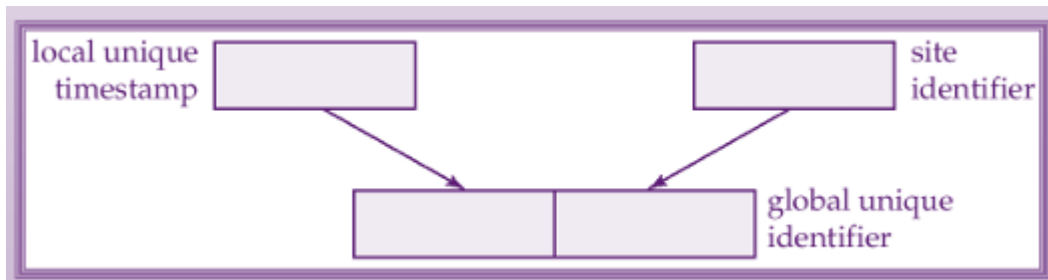
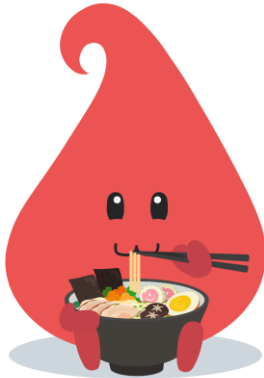
- إعطاء ختم وحيد لكل مناقلة.
- ترتيب المناقلات زمنياً حسب الأختام.

المشكلة الأساسية:

- كيف نولد ختماً عاماً وحيداً في نظام موزع؟

الحل:

- كل موقع يولد:
 - ختماً محلياً باستخدام:
 - عداد منطقي
 - أو ساعة محلية
- يُدمج الختم المحلي مع:
 - معرف الموقع
- فنحصل على:
 - ختم عام وحيد ومميز



نصبح الإنسان يُقاس بقدرته على تحمل التعقيد دون إنكار

معالجة الاستعلام الموزعة

Distributed Query Processing

الفكرة العامة:

في الأنظمة المركزية، تقاس **كلفة معالجة الاستعلام** أساساً بـ:

▪ عدد مرات الوصول إلى القرص.

أما في **الأنظمة الموزعة**، فيجب إضافة عوامل جديدة إلى العامل السابق أهمها:

1. كلفة نقل المعطيات عبر الشبكة.

2. التوازي

▪ معالجة أجزاء من الاستعلام في عدة مواقع في الوقت نفسه

3. تحسين الأداء (الربح) الناتج عن هذا التوازي

لذلك تصبح معالجة الاستعلام في النظام الموزع **أعقد** ولكن يمكن أن تكون **أسرع** إذا أحسن تنظيمها.

تحويل الاستعلام

Query Transformation

تمر معالجة الاستعلام الموزعة بثلاث مراحل رئيسية:

1. التفكيك Decomposition

نحول الاستعلام إلى:

▪ شجرة جبر علائقي مبنية على العلاقات الأصلية

الهدف:

▪ التعبير عن الاستعلام بشكل رسمي قابل للتحويل والتحسين.

2. التوطين Localization

استبدال العلاقات في شجرة الجبر بـ:

▪ أجزاء العلاقات (Fragments)

بمعنى آخر: تحويل شجرة الاستعلام الجبرية بحيث تعتمد على أجزاء الجداول (Relation Fragments) بدلاً من الجداول الكاملة.

الهدف:

▪ معرفة أين توجد المعطيات فعلياً

▪ وتقليل نقل المعطيات بين المواقع.

3. التحسين Optimization

بعد معرفة شكل الاستعلام ومكان المعطيات ينتقل النظام الى:

- إنتاج خطة تنفيذ الاستعلام (Query Plan) بأقل كلفة ممكنة (أفضل خطة تنفيذ)

قواعد التحسين الأساسية:

1. البدء بشجرة الاستعلام
 2. استبدال العلاقات بأجزائها
 3. دفع:
 4. حذف العمليات غير الضرورية
- عمليات الإسقاط (π) والاختيار (σ) إلى الأسفل
 - ورفع الاتحاد ($\cup$) إلى الأعلى

مثال أساسي (تفكيك الاستعلام):

الاستعلام:

```
SELECT R.A, S.D
FROM R, S
WHERE (R.B = 1 AND S.C = 2) AND (R.A = S.A)
```

تفاصيل الاستعلام:

هذا الاستعلام يطلب:

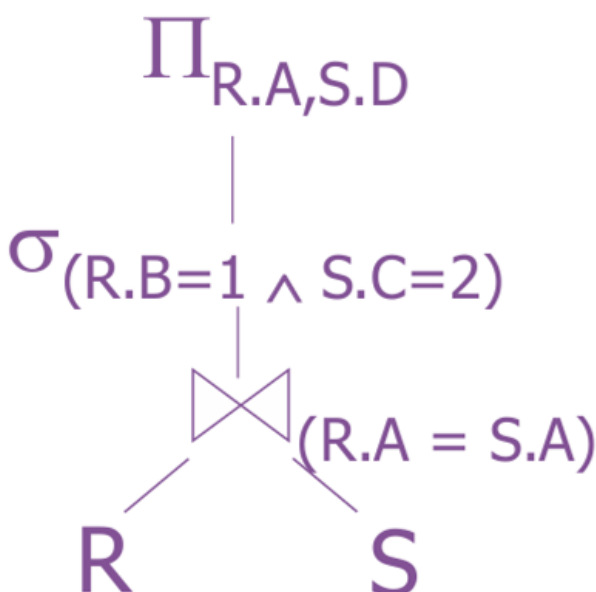
- العمود A من العلاقة R
- العمود D من العلاقة S

بشرطين:

- $R.B = 1$
- $S.C = 2$
- وجود وصل بين R و S على العمود A

التعبير عن الاستعلام بالجبر العلائقي:

- إسقاط: $\pi_{R.A, S.D}$
- اختيار: $\sigma_{(R.B = 1 \wedge S.C = 2)}$
- ربط: $R \bowtie_{(R.A = S.A)} S$



■ هذه مرحلة Decomposition

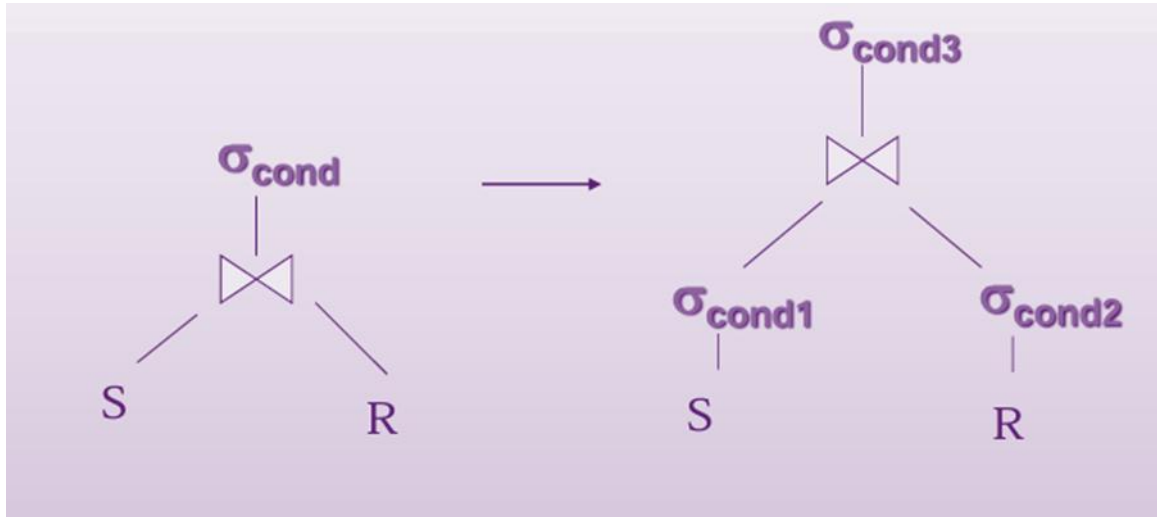
(مرحلة يتم فيها تحويل الاستعلام إلى شجرة جبر علائقي)

توزيع عملية الاختيار:

بدلاً من تطبيق الشرط بعد الدمج، يتم دفع الشروط إلى أسفل الشجرة

مثلاً:

$$\sigma_{\text{cond}}(R) \Rightarrow \sigma_{\text{cond}}(R1) \cup \sigma_{\text{cond}}(R2)$$



هذا يقلل حجم النتائج الوسيطة (تخفيف عدد الأسطر المطلوب دمجها).

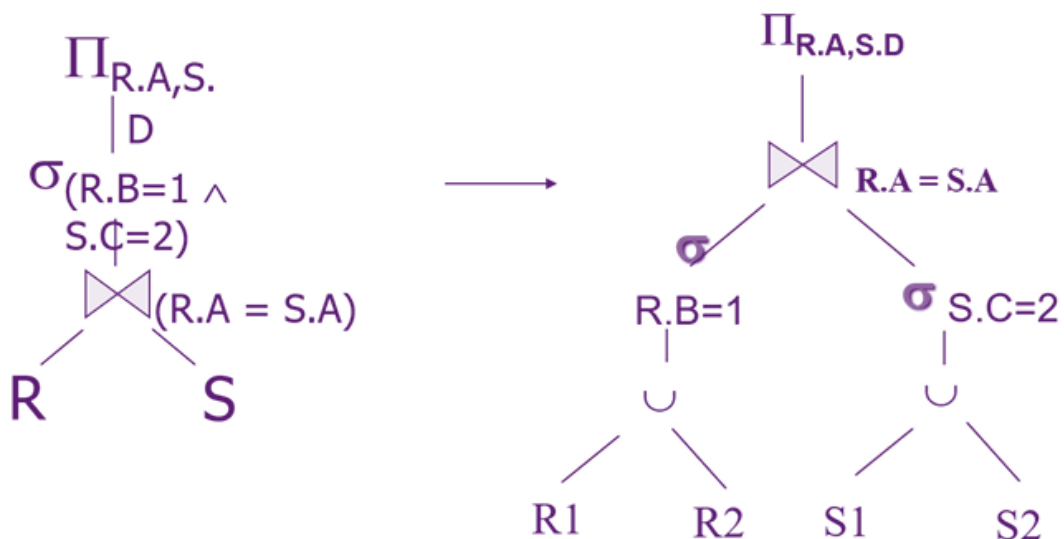
عند وجود تجزئة العلاقات:

■ يتم تطبيق عملية الاختيار محلياً على كل جزء.

وبالتطبيق على مثالنا: باعتبار العلاقات مجزأة (Fragmented)

■ $R1, R2 = R$

■ $S1, S2 = S$



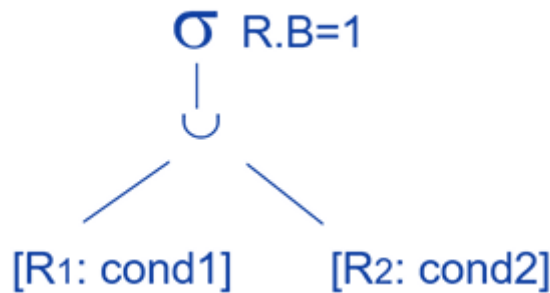
⚡ للتعبير عن تجزئة العلاقة:

نستخدم الصيغة التالية: [R : cond]

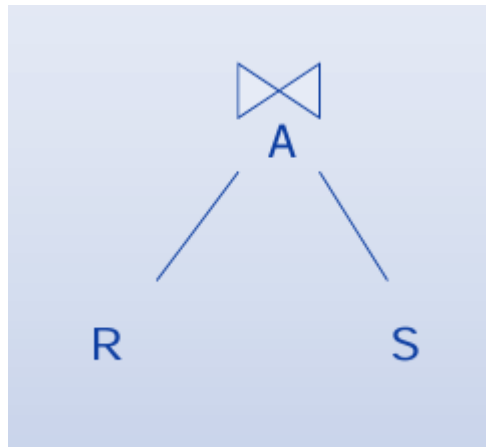
- [R1 : cond1] تعبر عن R مجزأة وفق الشرط cond1
- [R2 : cond2] تعبر عن R مجزأة وفق الشرط cond2

أي:

- R مجزأة وفق شروط محددة
- كل جزء يمثل مجالاً معيناً من المعطيات.



مثال يوضح التجزئة الأفقية للعلاقيتين R و S اعتماداً على خاصية مشتركة بينهما هي A:



تجزئة أفقية:

▪ [R1: A < 5]

▪ [R2: 5 ≤ A ≤ 10]

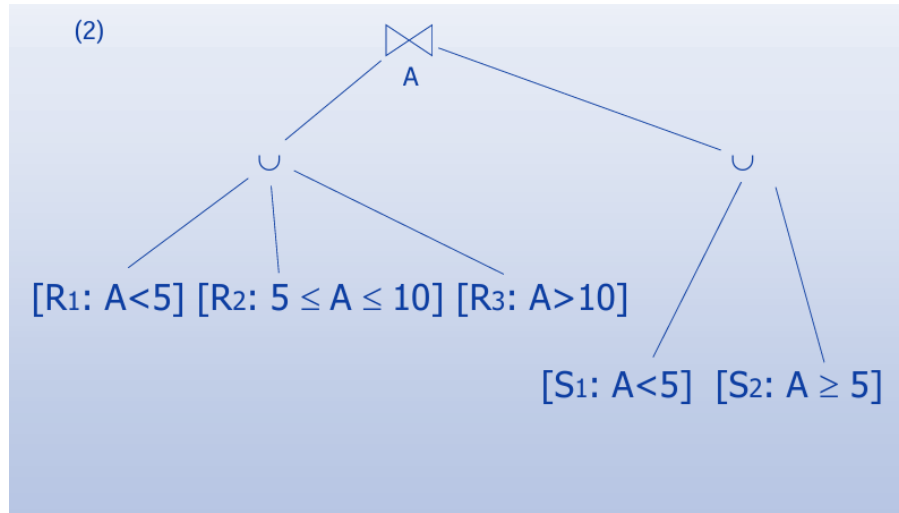
▪ [R3: A > 10]

و:

▪ [S1: A < 5]

▪ [S2: A ≥ 5]

الجهل ليس أن لا تعرف، بل أن ترفض أن تتعلم



ربط العلاقات المجزأة:

بدلاً من ربط R مع S كاملاً:

النظام يريد معرفة:

- أي الأجزاء يمكن أن تدمج معاً؟
 - وأي الأجزاء لا يمكن أن تنتج أي صفوف؟
- وهذا يعتمد على القيم الممكنة للخاصية A.

عند توليد جميع التركيبات بين أجزاء R وأجزاء S، سنجد أن:

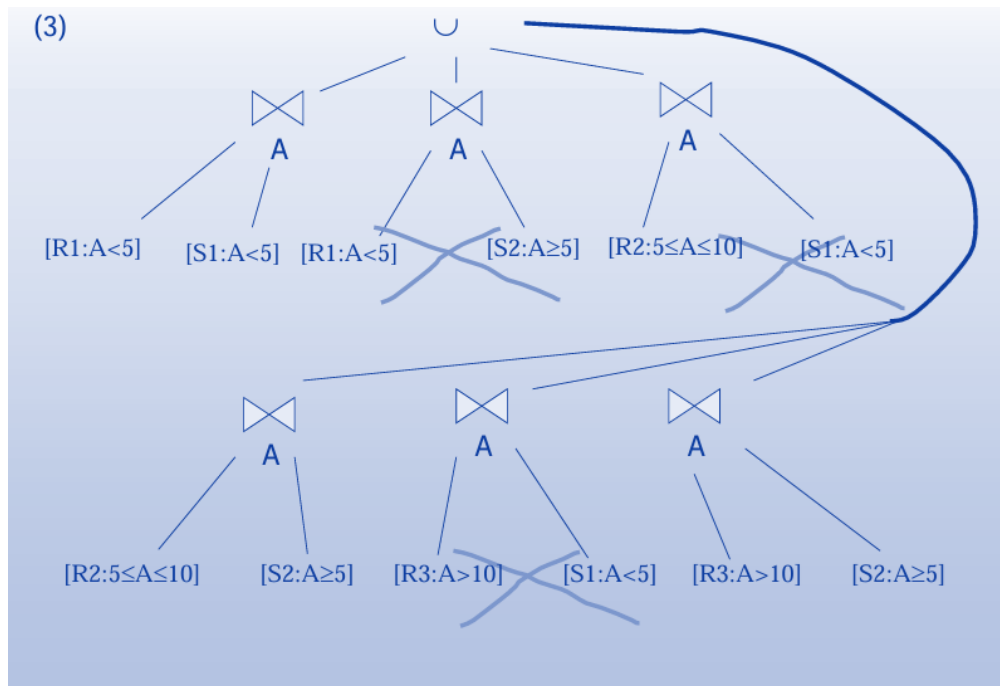
- بعض التركيبات منطقية ويمكن أن تنتج صفوفاً عند الدمج
- وبعضها غير منطقي لأن شروط التجزئة تجعل القيم غير متوافقة

مثال:

S2: $A \geq 5$

R1: $A < 5$

دمجهما غير منطقي



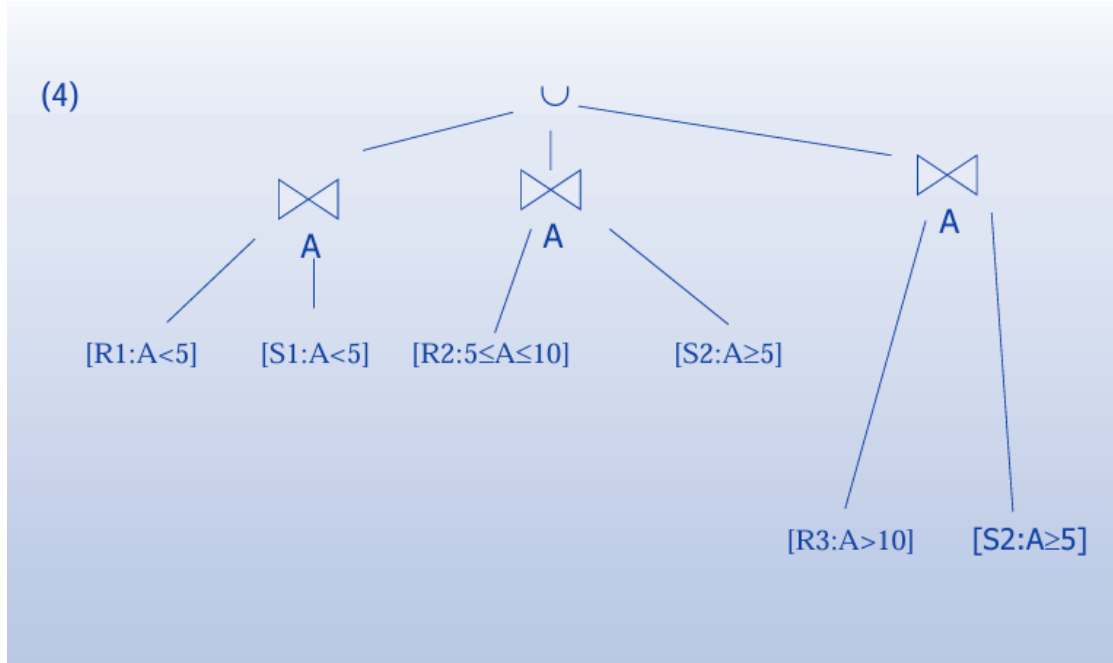
وبالتالي نربط فقط الأجزاء المتوافقة من حيث المجال.

مثلاً:

S2 مع R3

S2 مع R2

S1 مع R1



هذا يقلل كلفة النقل والمعالجة.

مثال على التجزئة الشجرية (الدمج بين التجزئة الأفقية والعمودية):

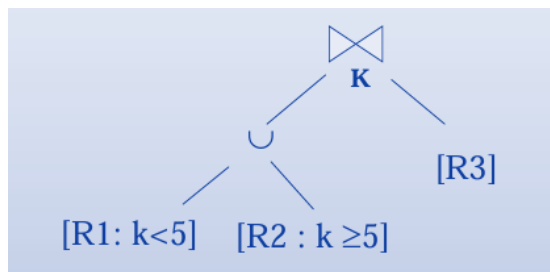
العلاقة R مجزأة إلى ثلاثة أجزاء:

$$R1 = \sigma k < 5 (\pi k, A R)$$

$$R2 = \sigma k \geq 5 (\pi k, A R)$$

$$R3 = \pi k, B R$$

التمثيل الشجري للتجزئة:



هذا يعني:

تم أولاً تقسيم العلاقة R إلى تجزئة أفقية حسب قيمة k (R1) و (R2)

ثم تم إنشاء جزء عمودي مستقل R3 يحتوي على الأعمدة (k, B)

مثال تطبيقي العلاقة (account):

التجزئة أفقية:

$$\text{account1} = \sigma \text{ branch-name} = \text{"Hillside"} (\text{account})$$

$$\text{account2} = \sigma \text{ branch-name} = \text{"Valleyview"} (\text{account})$$

الاستعلام المطلوب:

$$\sigma \text{ branch-name} = \text{"Hillside"} (\text{account})$$

بما أن العلاقة account مجزأة إلى account1 و account2، فإن النظام سيحوّل الاستعلام إلى:

$$\sigma \text{ branch-name} = \text{"Hillside"} (\text{account1} \cup \text{account2})$$

بعد ال localization يختزل الاستعلام إلى:

$$\sigma \text{ branch-name} = \text{"Hillside"} (\text{account1}) \cup \sigma \text{ branch-name} = \text{"Hillside"} (\text{account2})$$

لكن:

- account2 يحتوي فقط Valleyview ← ناتج الاختيار عليه مجموعة خالية

النتيجة النهائية:

account1 المطلوب فقط، أي:

$$\sigma \text{ branch-name} = \text{"Hillside"} (\text{account1})$$

- لا داعي للوصول إلى account2

- ولا إلى العلاقة الأصلية account

معالجة عملية الدمج البسيطة

نفترض وجود ثلاث علاقات:

- account

- depositor

- branch

وكل علاقة مخزنة في موقع مختلف:

- S1

- S2

- S3

عند تنفيذ استعلام يتطلب دمج العلاقات الثلاث:

$$\text{account} \bowtie \text{depositor} \bowtie \text{branch}$$

بعد تنفيذ الدمج بين الأجزاء المختلفة النظام يحتاج إلى إظهار النتيجة في نفس الموقع Si الذي أُلْقِيَ الاستعلام منه.

الاستراتيجيات الممكنة لمعالجة الاستعلام

Possible Query Processing Strategies

الفكرة العامة:

عند تنفيذ استعلام يحتاج إلى دمج ثلاث علاقات:

- account
- depositor
- branch

وكانت هذه العلاقات مجزأة وموزعة على مواقع مختلفة، فإن النظام يحتاج إلى اختيار أفضل استراتيجية لمعالجة الاستعلام.

الاستراتيجية الأولى: إرسال كل العلاقات إلى موقع واحد (SI)

اختيار استراتيجية معالجة الدمج محلياً في موقع SI بإرسال نسخة من العلاقات الثلاث إليه أي:

1. إرسال account إلى الموقع SI
2. إرسال depositor إلى الموقع SI
3. إرسال branch إلى الموقع SI
4. تنفيذ عملية الدمج محلياً في SI

الاستراتيجية الثانية: تنفيذ الدمج على مراحل بين المواقعمثلاً:

1. إرسال نسخة من account إلى موقع S2
2. في S2 يتم دمج account مع depositor ← النتيجة تسمى temp1
3. إرسال temp1 إلى موقع S3
4. في S3 يتم دمج temp1 مع branch ← النتيجة تسمى temp2
5. إرسال temp2 إلى الموقع SI لعرض النتيجة النهائية

يمكن اعتبار استراتيجيات مختلفة أخرى بتبادل الأدوار في المواقع:

أي:

- يمكن تنفيذ الدمج الأول في S3 بدلاً من S2
 - أو تنفيذ الدمج النهائي في موقع آخر
 - أو اختيار موقع يحتوي على أقل كمية معطيات منقولة.
- هذه مرونة يعتمد عليها النظام لاختيار أفضل خطة.

التقدم البسيط اليوم، أفضل من نية عظيمة بلا فعل

ما العوامل التي يجب أخذها بالاعتبار عند اختيار أفضل استراتيجية؟

1. كمية البيانات المرسل:

■ كلما كانت البيانات أقل، كانت الاستراتيجية أفضل

2. تكلفة نقل البيانات بين المواقع:

■ النقل عبر الشبكة مكلف جداً في الأنظمة الموزعة

3. سرعة المعالجة في كل موقع:

■ بعض المواقع قد تكون أبطأ أو مشغولة

■ لذلك اختيار الموقع المناسب مهم

The End...

إذا كنت تقرأ هذه السطور... فاعلم أنك لم تنم المقرر فقط،

بل أثبت أنك شخص لا يهرب أمام 36 صفحة،

ولا يتراجع أمام 11 محاضرة،

ولا يخاف من قواعد المعطيات

